

Лабораторная работа №13

Дрекина Арина Андреевна

2026.05.08

Российский университет дружбы народов, Москва, Россия

Докладчик

- Дрекина Арина Андреевна
- студентка факультета физико-математических и естественных наук
- Российский университет дружбы народов им. П. Лумумбы
- 1032253548@rudn.ru

Изучить основы программирования в оболочке ОС UNIX. Научится писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

1. Используя команды `getopts` `grep`, написать командный файл, который анализирует командную строку с ключами: — `-iinputfile` — прочитать данные из указанного файла; — `-ooutputfile` — вывести данные в указанный файл; — `-р` шаблон — указать шаблон для поиска; — `-C` — различать большие и малые буквы; — `-n` — выдавать номера строк. а затем ищет в указанном файле нужные строки, определяемые ключом `-р`.

2. Написать на языке Си программу, которая вводит число и определяет, является ли оно больше нуля, меньше нуля или равно нулю. Затем программа завершается с помощью функции `exit(n)`, передавая информацию в о коде завершения в оболочку. Командный файл должен вызывать эту программу и, проанализировав с помощью команды `$?`, выдать сообщение о том, какое число было введено.

3. Написать командный файл, создающий указанное число файлов, пронумерованных последовательно от 1 до n (например 1.tmp, 2.tmp, 3.tmp, 4.tmp и т.д.). Число файлов, которые необходимо создать, передаётся в аргументы командной строки. Этот же командный файл должен уметь удалять все созданные им файлы (если они существуют).

4. Написать командный файл, который с помощью команды `tar` запаковывает в архив все файлы в указанной директории. Модифицировать его так, чтобы запаковывались только те файлы, которые были изменены менее недели тому назад (использовать команду `find`).

Командный процессор — это программа, позволяющая пользователю взаимодействовать с операционной системой компьютера. В операционных системах типа UNIX/Linux наиболее часто используются следующие реализации командных оболочек: – оболочка Борна — стандартная командная оболочка UNIX/Linux, содержащая базовый, но при этом полный набор функций;

– С-оболочка — надстройка на оболочкой Борна, использующая С-подобный синтаксис команд с возможностью сохранения истории выполнения команд;

– оболочка Корна — напоминает оболочку С, но операторы управления программой совместимы с операторами оболочки Борна;

– BASH — сокращение от Bourne Again Shell, в основе своей совмещает свойства оболочек С и Корна.

Оболочка bash поддерживает встроенные арифметические функции. Команда let является показателем того, что последующие аргументы представляют собой выражение, подлежащее вычислению.

например:

```
$ let x=5 $ while > (( x-=1 )) > do > something > done
```

Весьма необходимой при программировании является команда getopts, которая осуществляет синтаксический анализ командной строки, выделяя флаги, и > для объявления переменных. Синтаксис команды следующий:

```
getopts option-string variable [arg ... ]
```

В обобщённой форме оператор цикла for выглядит следующим образом:

```
for имя [in список-значений] do список-команд done
```

Оператор выбора case реализует возможность ветвления на произвольное число ветвей.Пример:

```
case имя in шаблон1) список-команд;; шаблон2) список-команд;; ... esac
```

В обобщённой форме условный оператор if выглядит следующим образом:

```
if список-команд then список-команд {elif список-команд then список-команд}  
[else список-команд] fi
```

В обобщённой форме оператор цикла `while` выглядит следующим образом:

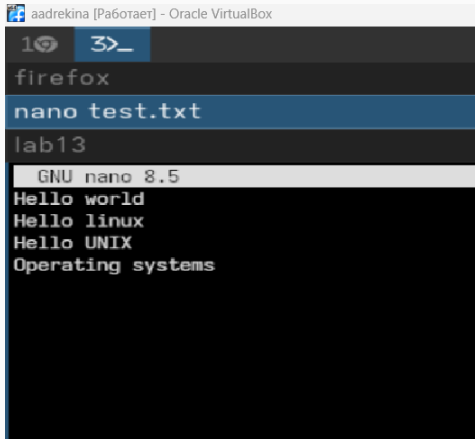
```
while список-команд do список-команд done
```

Два несложных способа позволяют вам прерывать циклы в оболочке `bash`. Команда `break` завершает выполнение цикла, а команда `continue` завершает данную итерацию блока операторов. Пример бесконечного цикла `while` с прерыванием в момент, когда файл перестает существовать:

```
while true do if [! -f $file] then break fi sleep 10 done
```

Выполнение лабораторной работы

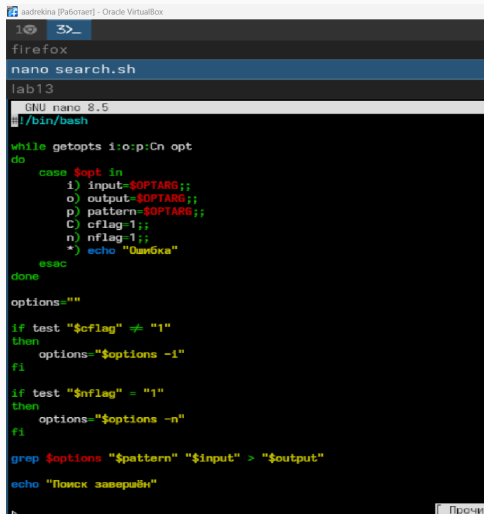
Создание текстового файла и ввод программы.



```
aadrekina [Работает] - Oracle VirtualBox
1 3>_
firefox
nano test.txt
lab13
GNU nano 8.5
Hello world
Hello linux
Hello UNIX
Operating systems
```

Рис. 1: Создание файл и ввод теста.

Создание еще одного файла для программы.



```
1 3>_
firefox
nano search.sh
lab13
GNU nano 8.5
#!/bin/bash

while getopts i:op:Cn opt
do
    case $opt in
        i) input=$OPTARG;;
        o) output=$OPTARG;;
        p) pattern=$OPTARG;;
        C) cflag=1;;
        n) nflag=1;;
        *) echo "Ошибка"
        ;;
    esac
done

options=""

if test "$cflag" != "1"
then
    options="$options -i"
fi

if test "$nflag" = "1"
then
    options="$options -n"
fi

grep $options "$pattern" "$input" > "$output"

echo "Поиск завершен"
```

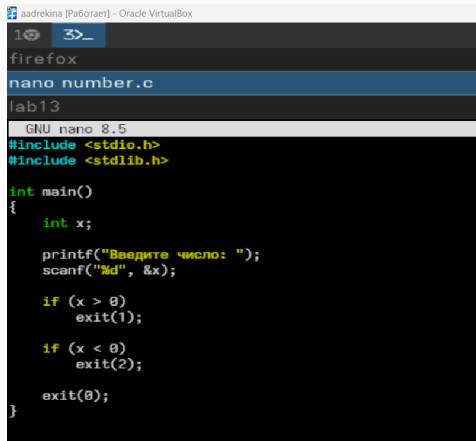
Рис. 2: Код программы.

Запуск программы.

```
madrekin@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab1$ cat result.txt  
1:Hello world  
2:Hello linux  
3:Hello UNIX  
madrekin@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab1$ |
```

Рис. 3: Изменения прав,запуск.

Ввод кода на си.



```
1 aadrekina [Работает] - Oracle VirtualBox
3>_
firefox
nano number.c
lab13
GNU nano 8.5
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int x;

    printf("Введите число: ");
    scanf("%d", &x);

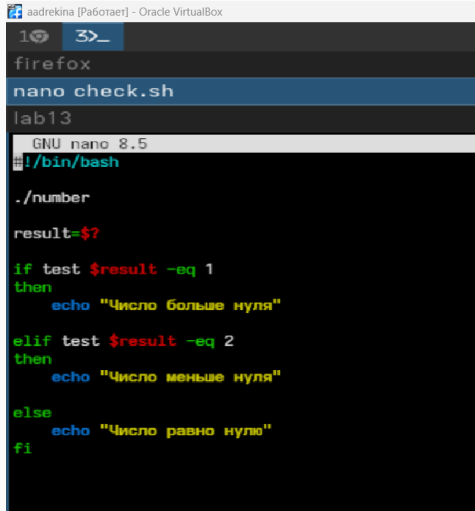
    if (x > 0)
        exit(1);

    if (x < 0)
        exit(2);

    exit(0);
}
```

Рис. 4: Ввод программы

Еще один код.



```
aadrekina [Работает] - Oracle VirtualBox
1 3>_
firefox
nano check.sh
lab13
GNU nano 8.5
#!/bin/bash

./number

result=${?}

if test $result -eq 1
then
    echo "Число больше нуля"

elif test $result -eq 2
then
    echo "Число меньше нуля"

else
    echo "Число равно нулю"
fi
```

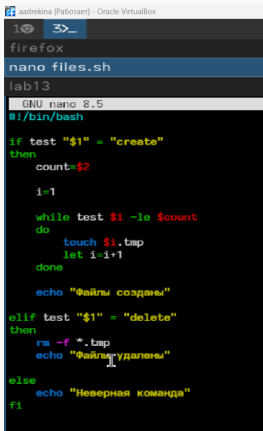
Рис. 5: Ввод программы.

Запуск программы.

```
adrek1na@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab1$ ./check.sh
adrek1na@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab1$ ./check.sh
Введите число: -5
Число меньше нуля
adrek1na@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab1$ |
```

Рис. 6: Запуск программы и вывод.

Новый код программы



```
aadrekina [Рабочий] - Oracle VM VirtualBox
1 3>_
firefox
nano files.sh
lab13
GNU nano 8.5
#!/bin/bash

if test "$1" = "create"
then
    count=$2

    i=1

    while test $i -le $count
    do
        touch $i.tmp
        let i=i+1
    done

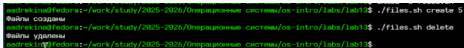
    echo "Файлы созданы"

elif test "$1" = "delete"
then
    rm -f *.tmp
    echo "Файлы удалены"

else
    echo "Неверная команда"
fi
```

Рис. 7: Ввод программы.

Запуск программы.

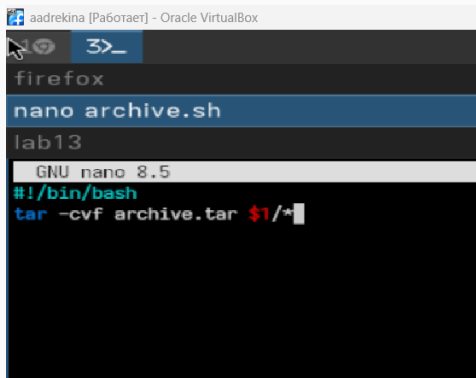
A terminal window with a black background and green text. The prompt is 'madrekina@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab13\$'. The first command is './files.sh create 5', followed by the output 'Файлы созданы'. The second command is './files.sh delete', followed by the output 'Файлы удалены'. The prompt appears again at the end.

```
madrekina@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab13$ ./files.sh create 5
Файлы созданы
madrekina@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab13$ ./files.sh delete
Файлы удалены
madrekina@fedora:~/work/study/2025-2026/Операционные системы/os-intro/labs/lab13$
```

Рис. 8: Запуск программы и удаление этих же файлов.

Создалось то количество текстовых файлов, которое я ввела и удалилось.

Создание еще одного файла и ввод кода

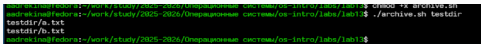


The screenshot shows a terminal window titled 'aadrekina [Работает] - Oracle VirtualBox'. The terminal has a dark background with a blue prompt '3>_'. The command history shows 'firefox', 'nano archive.sh', and 'lab13'. The current session is running 'GNU nano 8.5'. The first line of the file is '#!/bin/bash'. The second line contains the command 'tar -cvf archive.tar \$1/*' with a red cursor at the end.

```
aadrekina [Работает] - Oracle VirtualBox
3>_
firefox
nano archive.sh
lab13
GNU nano 8.5
#!/bin/bash
tar -cvf archive.tar $1/*
```

Рис. 9: Создание файлов и ввод кода.

Запуск программы

A terminal window with a black background and green text. The prompt is 'root@kali:~/os-intro/1abs/1ab1\$'. The user enters 'chmod +x archive.sh'. The prompt changes to 'root@kali:~/os-intro/1abs/1ab1\$'. The user enters './archive.sh testdir'. The output shows 'testdir/a.txt' and 'testdir/b.txt' on two separate lines. The prompt returns to 'root@kali:~/os-intro/1abs/1ab1\$'.

```
root@kali:~/os-intro/1abs/1ab1$ chmod +x archive.sh
root@kali:~/os-intro/1abs/1ab1$ ./archive.sh testdir
testdir/a.txt
testdir/b.txt
root@kali:~/os-intro/1abs/1ab1$
```

Рис. 10: Запускаем программу, получаем вывод.

В ходе выполнения лабораторной работы мы изучили основы программирования в оболочке ОС UNIX. Научились писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.